

Abstract Data Types

Algorithms and Data Structures

Abstract Data Types

- ADTs Revisited
- Generics – Java Issues
- ADT Implementation
- Linked Structures
- ADT Commonalities

Purpose

- Behavioural contract with user
- Encapsulation
 - Implementation details are hidden
- Isolation
 - User code and ADT implementation can both be changed without affecting each other
- Flexibility
 - Alternative implementations are interchangeable
 - Optimized for specific purpose

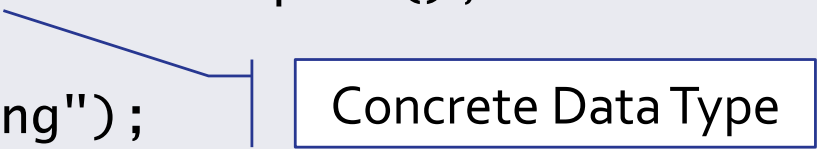
Stack ADT

```
1 package neu.adt;
2
3 import java.util.NoSuchElementException;
4
5 public interface Stack<T> {
6     public void    push(T element);
7     public T       pop() throws NoSuchElementException;
8
9     public T       top() throws NoSuchElementException;
10    public int      size();
11    public boolean  isEmpty();
12 }
```

// how would you implement top() in terms of already implemented methods push() and pop() ?

Using Stack ADT

```
1  public class Example {  
2  
3      public static void main(String[] args) {  
4          Stack<String> stack = new StackImpl<>();  
5          stack.push("A String");  
6          stack.push("Another String");  
7  
8          String str = stack.pop();  
9      }  
10 }
```



Concrete Data Type

During Compilation... Treated As

```
1 package neu.adt;  
2  
3 import java.util.NoSuchElementException;  
4  
5 public interface Stack {  
6  
7     public void push(String element);  
8     public String pop() throws NoSuchElementException;  
9  
10    public String top() throws NoSuchElementException;  
11    public int size();  
12    public boolean isEmpty();  
13 }
```

Implementation

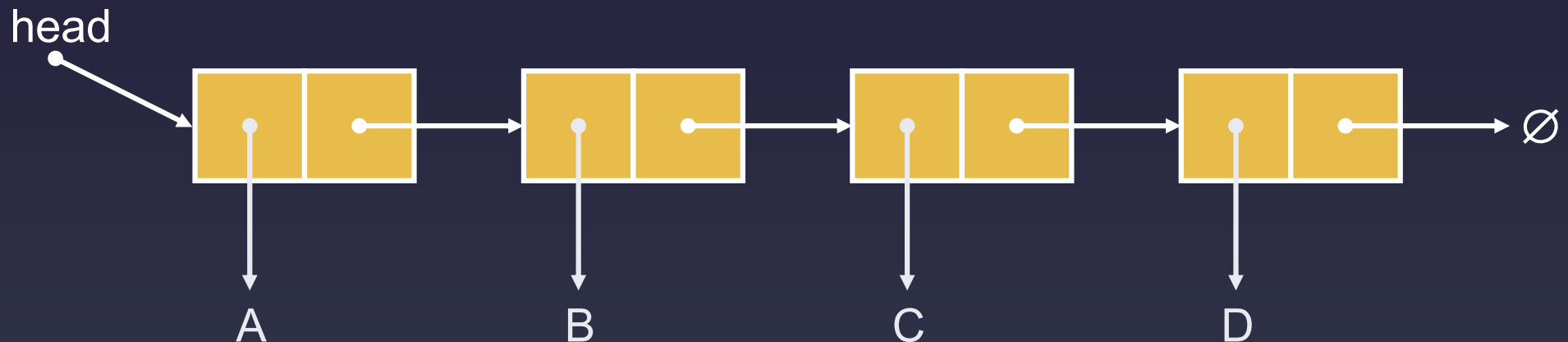
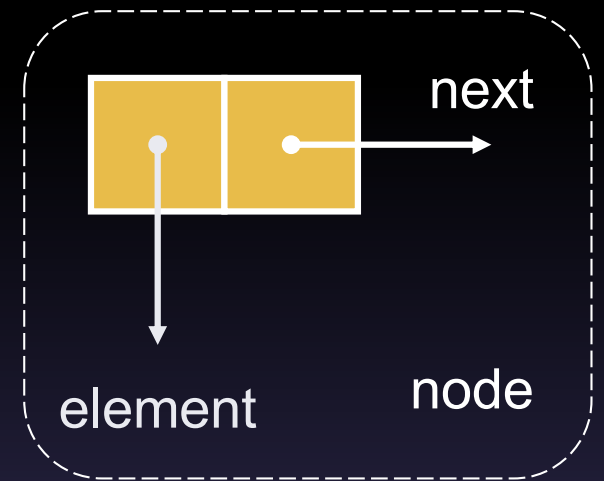
- Remember that an Interface has to be implemented – a design decision
- ADTs are implemented using **arrays** or **linked** data structures
 - Many ADTs may be implemented with either
 - Some ADTs are implemented using a combination of both
- ADTs are often implemented using other ADTs

Common Linked Data Structures

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Tree

Singly Linked List

- Concrete data structure
 - sequence of nodes
 - head pointer
- Nodes store
 - element
 - link to next node



Linked List Implementation

```
class ListNode<T> {  
    T element;  
    ListNode<T> next;  
}  
  
public class LinkedList<T> {  
    private ListNode<T> head;  
    private ListNode<T> tail;  
    private int size;  
  
    ...  
}
```

```
// we have seen this earlier
```

Linked List Implementation

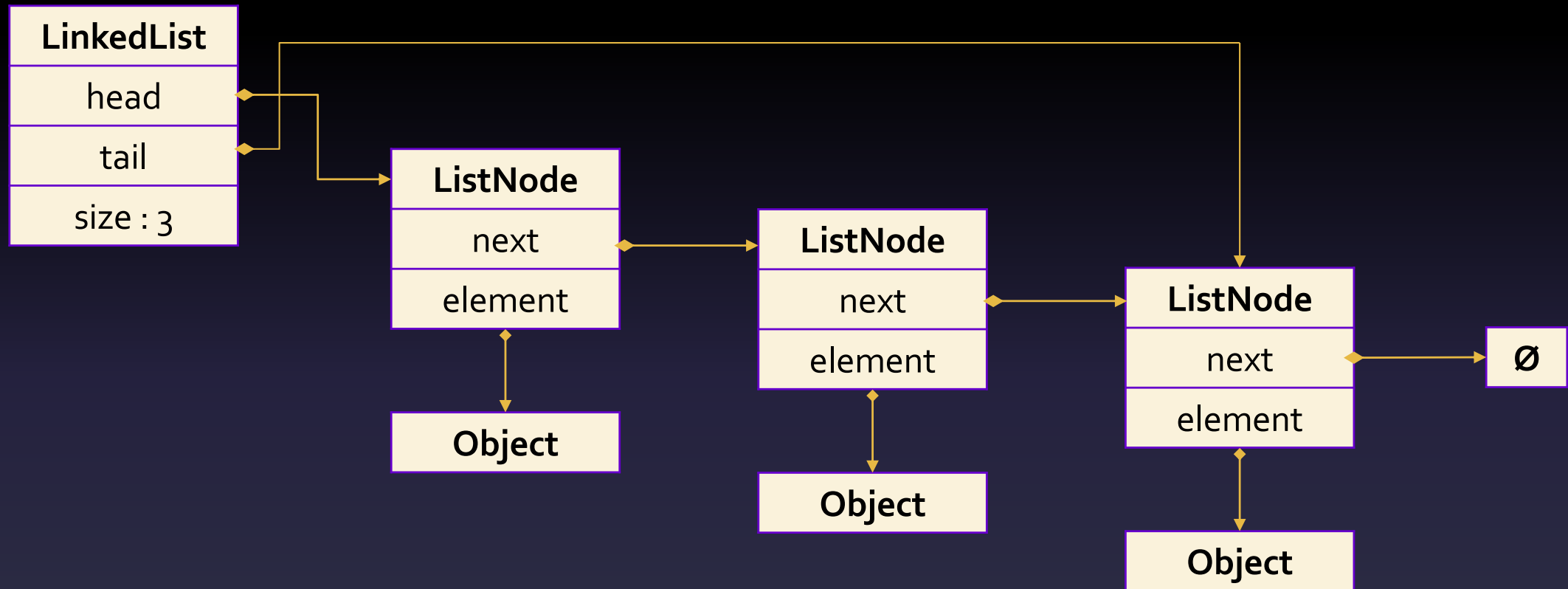
```
public void addFirst(ListNode<T> aNode) {  
    aNode.next = this.head;  
    this.head = aNode;           // we have seen this earlier  
  
    if (this.tail == null) {  
        this.tail = aNode;  
    }  
    this.size++;  
}
```

```
public void addLast(ListNode<T> aNode) {  
    aNode.next = null;  
    this.tail.next = aNode;  
    this.tail = aNode;  
  
    if (this.head == null) { // here if added to empty list  
        this.head = aNode;  
    }  
    this.size++;  
}
```

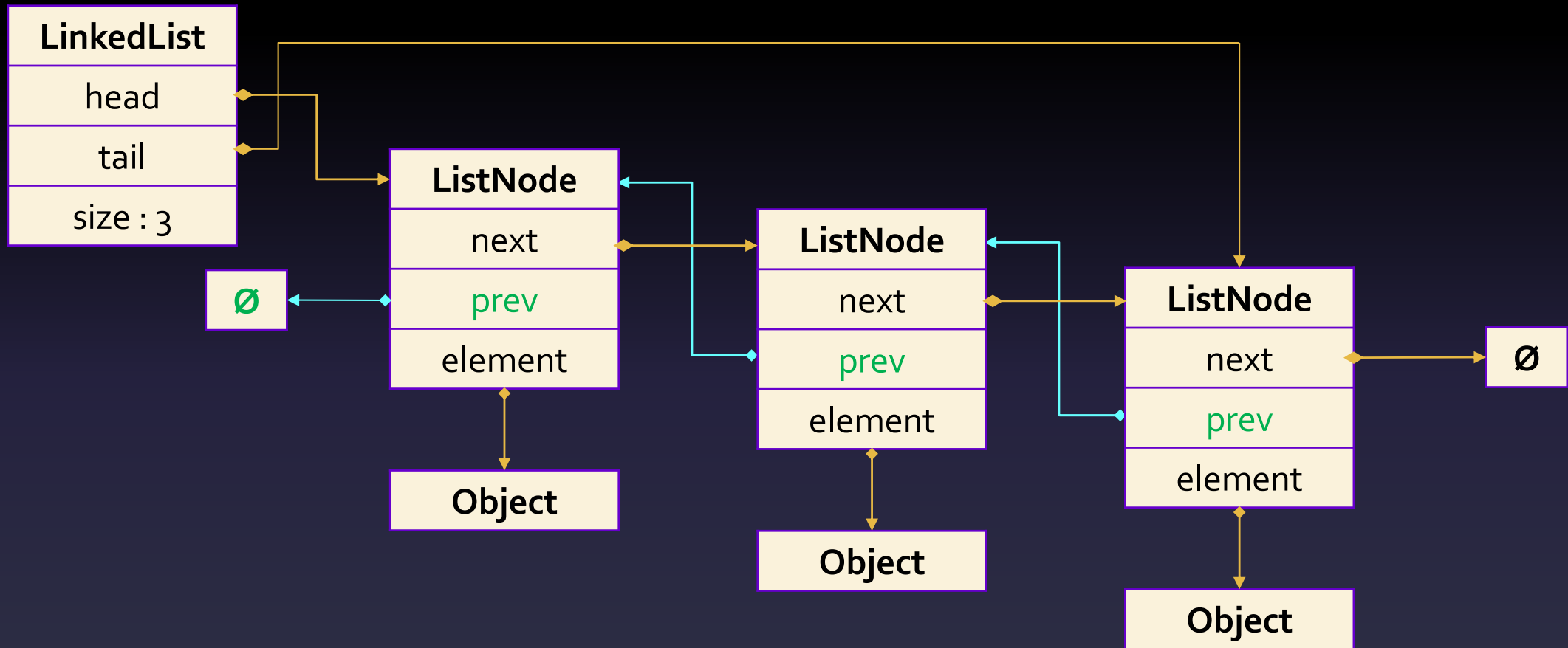
Linked List Implementation

```
public T removeFirst() { // returns an element
    if (this.size == 0) {
        return null; // Special case of list is empty.
    }
    T elementToReturn = this.head.element; // save element
    this.head = this.head.next; // Move head ptr to next node in list.
    this.size--;
    if (this.size == 0) {
        this.tail = null; // List is now empty, set tail to null.
    }
    return elementToReturn; // returns saved removed element
}
```

Singly Linked List



Doubly Linked List



Bidirectional Traversal:

- **Doubly Linked List:** Each node in a doubly linked list contains two pointers—one pointing to the next node and one pointing to the previous node. This allows for traversal in both forward and backward directions.
- **Singly Linked List:** A singly linked list only allows traversal in one direction (from the head to the end) because each node only has a pointer to the next node.
- **Advantage:** The ability to traverse in both directions is particularly useful in scenarios where you need to move back and forth between nodes, such as in navigating a browser's history for instance.

Bidirectional Traversal:

- **Easier Insertion Before a Given Node:**
- **Doubly Linked List:** Inserting a new node before a given node is simpler because you can easily access the previous node through the prev pointer and adjust the pointers accordingly.
- **Singly Linked List:** Inserting a node before a given node requires traversing the list to find the node that precedes the target node, which can be inefficient.
- **Advantage:** The ability to insert nodes before a given node efficiently makes doubly linked lists more flexible in certain applications, such as in implementing complex data structures like balanced trees or certain types of queues.

Reversing the List:

- **Doubly Linked List:** Reversing a doubly linked list is straightforward because each node has pointers to both its previous and next nodes.
- **Singly Linked List:** Reversing a singly linked list requires reassigning the next pointers of all nodes, which can be more complex and less efficient.
- **Advantage:** If reversing the list is a common operation, a doubly linked list can perform this operation more efficiently.

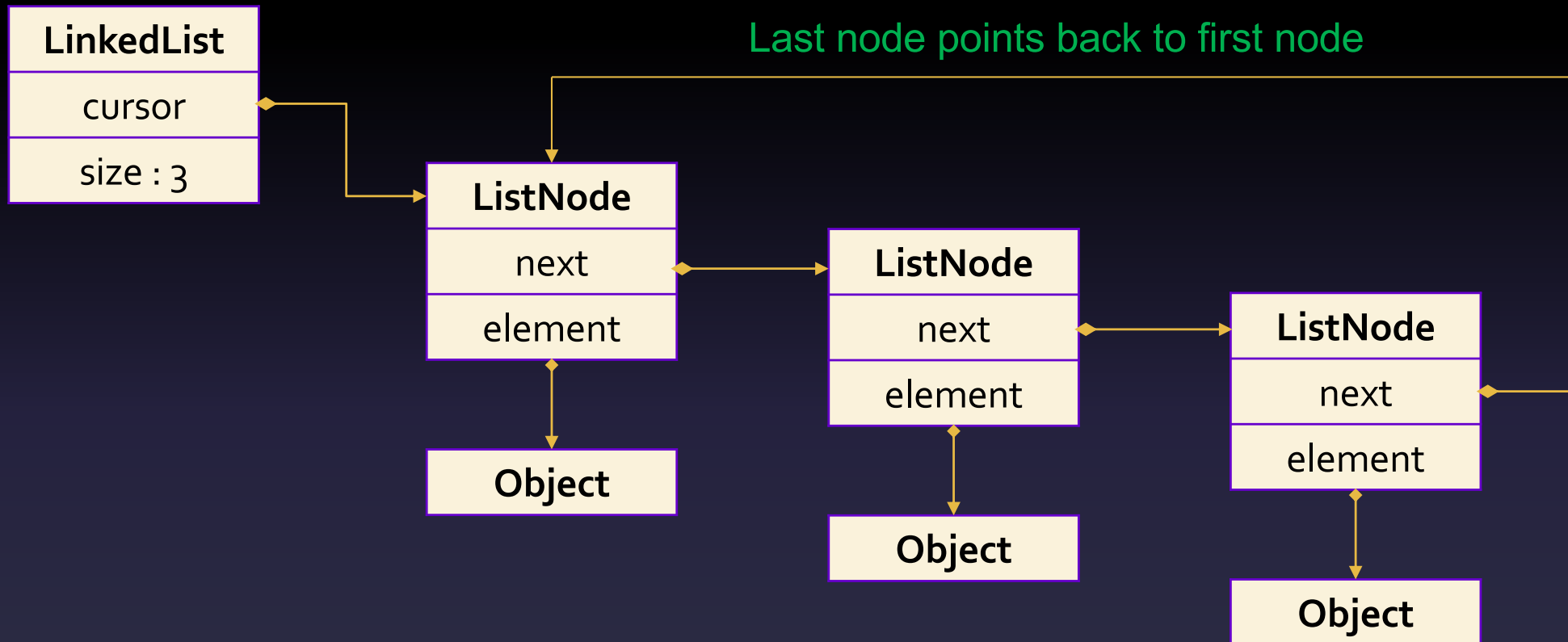
Efficient Implementation of Deques:

- **Doubly Linked List:** Deques (double-ended queues), which allow insertion and deletion at both ends, are naturally suited to doubly linked lists because of their bidirectional pointers.
- **Singly Linked List:** Implementing a deque with a singly linked list can be less efficient because it may require traversing the list to access the node at the opposite end.
- **Advantage:** Doubly linked lists provide a more natural and efficient implementation for data structures like deques, which require operations at both ends. (more on dequeues later)

Doubly Linked List Trade-offs:

- **Memory Overhead:** The primary disadvantage of a doubly linked list is that each node requires extra memory to store the additional pointer (to the previous node). This can be significant if memory is a concern, especially in environments with limited resources.
- **Complexity:** Maintaining a doubly linked list is also more complex, updating all pointers during insertions and deletions.

Circularly Linked List



Circularly Linked Lists:

- **Efficient Traversal from Any Node:** In a circularly linked list, you can start traversal from any node and eventually reach every other node in the list because the list is circular.
- This is particularly useful in applications where the list needs to be processed in a round-robin fashion (e.g., CPU scheduling in operating systems).

Circularly Linked Lists:

- **No Need for Special Cases When Inserting/Deleting Nodes:** In a traditional singly or doubly linked list, inserting or deleting the last node often requires special handling because it points to null. In a circular list, since the last node points back to the first node, the list operations can be implemented more uniformly.

Circularly Linked Lists:

- **Improved Handling of Circular Data**

Structures: Circularly linked lists are well-suited for applications that naturally involve circular structures, such as implementing a round-robin scheduler, a circular buffer, or navigating through a set of repeating elements (e.g., a carousel).

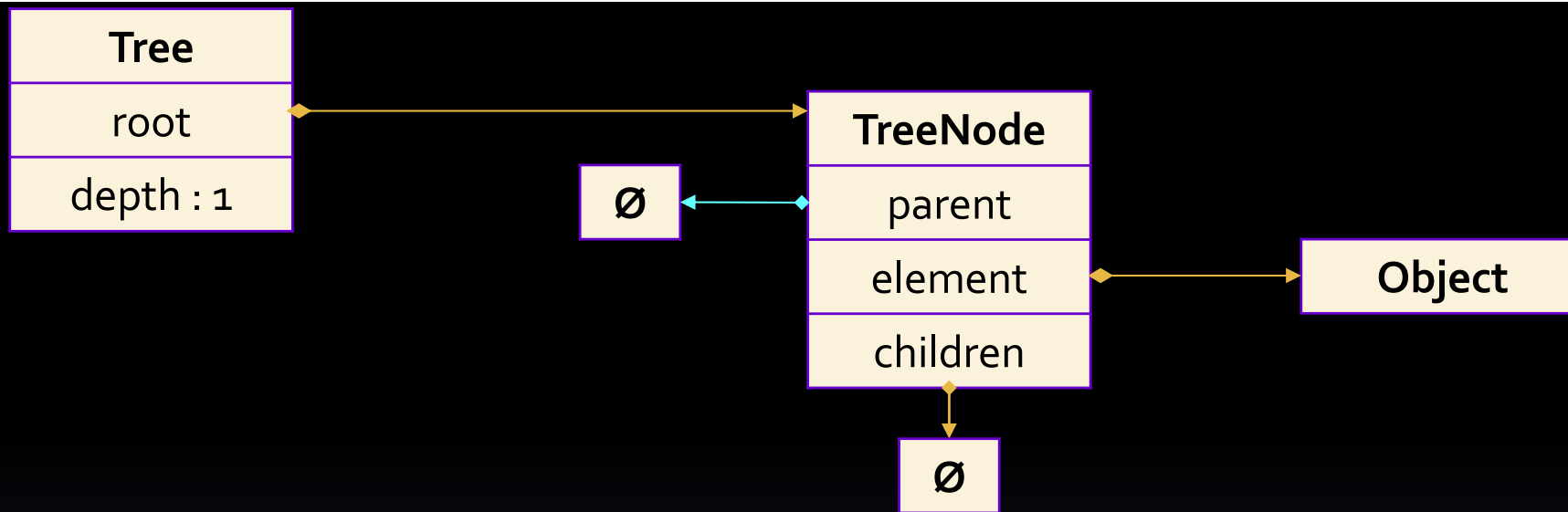
Circularly Linked Lists:

- **Efficient Queue Implementation:**
- Circularly linked lists can be efficiently used to implement queues, especially circular queues, without needing to move data around. The front and rear pointers can be managed easily without the risk of overflow as long as the list is not full.

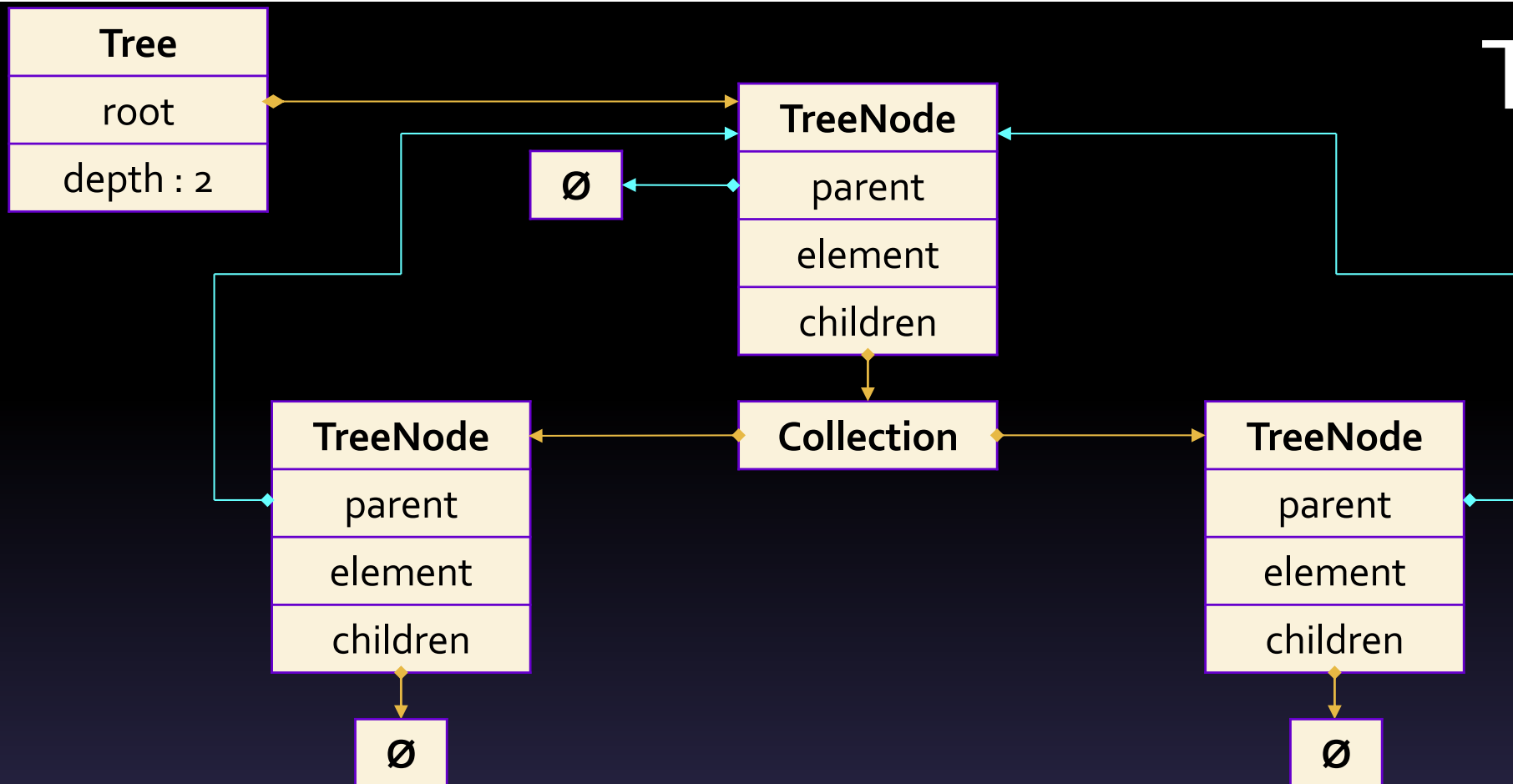
Tree



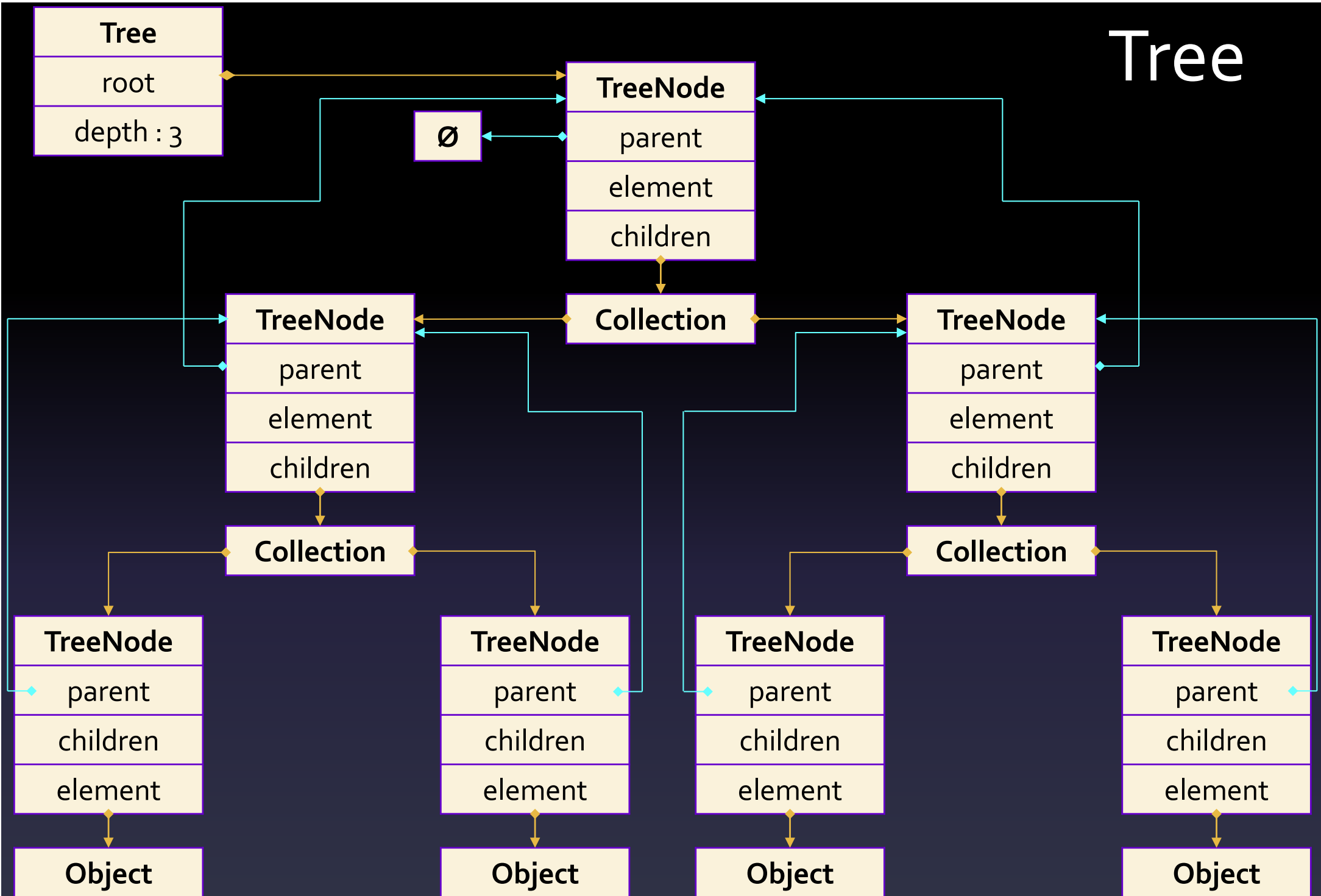
Tree



Tree



Tree



ADT Commonalities

- In the coming weeks we will look at how these data structures are used to implement common ADTs
- Common operations
 - Insertion/removal of data elements
 - Iteration/traversal of data elements
 - Search for specific data element

ADT Commonalities

ADT	Modification	Accessor	Iterators
Stack	push pop	top	
Queue	enqueue dequeue	front	
Deque	addLast removeFront addFront removeLast	getFirst getLast	
Tree	replace	root parent	children iterator
Map	put remove	get	keys values entries
Dictionary	insert remove	find	findAll entries
Set	union intersect subtract		

Lecture Summary

- ADTs provide behavioral contract and separation of implementation
- Specifying ADTs in Java using interfaces
- Generics
- Implementation
- ADT Commonalities